Task for BD06

Task for BD06
Farré Vicente Josep
Bases de dades

Using the **classicmodels** schema used in previous exercises, solve the following exercises:

Exercise 1

An employee cannot be inserted into the `employees` table if another employee with the title "Sales Manager" already exists in the same office. In other words, there can only be one "Sales Manager" per office (`officeCode`).
If a new employee with the title "Sales Manager" is inserted into an office that already has one, the trigger must throw an error with a message indicating that a "Sales Manager" already exists in that office. The trigger only prevents insertion and does not handle updates.

Examples:

3	20:27:32	CREATE TRIGGER before_insert_sales_manager BEFORE INSERT ON employees FOR EACH ROW ...	0 row(s) affected	0.016 sec
4	20:27:38	INSERT INTO employees (employeeNumber, lastName, firstName, extension, email, officeCode, reports...	Error Code: 1644. Only one Sales Manager is allowed per office	0.000 sec
5	20:27:51	INSERT INTO employees (employeeNumber, lastName, firstName, extension, email, officeCode, reports...	1 row(s) affected	0.000 sec

Problem:

Only one “Sales Manager” is allowed per office.

Solution:

A **BEFORE INSERT TRIGGER** on the employees table.

Explanation:

Before inserting a new employee:

- Check if the job title is *Sales Manager*
- Count how many already exist in that office
- If one already exists → raise an error

Code:

```
USE classicmodels;

DROP TRIGGER IF EXISTS before_insert_sales_manager;

DELIMITER $$

CREATE TRIGGER before_insert_sales_manager
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
    IF NEW.jobTitle = 'Sales Manager' THEN
        IF (SELECT COUNT(*)
            FROM employees
            WHERE jobTitle = 'Sales Manager'
            AND officeCode = NEW.officeCode) >= 1 THEN
```

Task for BD06
Farré Vicente Josep
Bases de dades

```
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'A Sales Manager already exists in this office';
        END IF;
    END IF;
END$$

DELIMITER ;

-- TESTS

-- FAIL
INSERT INTO employees
VALUES (3001, 'Smith', 'John', 'x111', 'john.smith@classicmodels.com', 1, 1002, 'Sales Manager');

-- SUCCESS
INSERT INTO employees
VALUES (3002, 'Brown', 'Laura', 'x112', 'laura.brown@classicmodels.com', 3, 1002, 'Sales Manager');

-- SUCCESS (non-manager)
INSERT INTO employees
VALUES (3003, 'White', 'Chris', 'x113', 'chris.white@classicmodels.com', 1, 1002, 'Sales Rep');
```

Exercise 2

A customer cannot have more than 3 active orders (statuses 'In Process', 'On Hold', or 'Shipped') at any given time. Create a trigger that checks this condition before inserting into the `orders` table, throwing an error with a custom message if the customer attempts to create a fourth active order. Demonstrate how to test the trigger.

Examples:

6	21:16:46	INSERT INTO orders (orderNumber, orderDate, requiredDate, shippedDate, status, customerNumber) V...	1 row(s) affected	0.016 sec
7	21:17:47	INSERT INTO orders (orderNumber, orderDate, requiredDate, shippedDate, status, customerNumber) V...	Error Code: 1644. A customer cannot have more than 3 active orders	0.000 sec

Problem:
A customer cannot have more than **3 active orders**.
Solution:
BEFORE INSERT trigger on `orders`.
Explanation:

- Count orders with status:
  - *In Process*
  - *On Hold*
  - *Shipped*
- If count ≥ 3 → error

```
DROP TRIGGER IF EXISTS before_insert_order;

DELIMITER $$

CREATE TRIGGER before_insert_order
BEFORE INSERT ON orders
FOR EACH ROW
BEGIN
    IF (
        SELECT COUNT(*)
        FROM orders
```

Task for BD06

Farré Vicente Josep

Bases de dades

```
        WHERE customerNumber = NEW.customerNumber
        AND status IN ('In Process', 'On Hold', 'Shipped')
    ) >= 3 THEN

        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'A customer cannot have more than 3 active orders';
    END IF;
END$$

DELIMITER ;

-- TESTS

-- SUCCESS
INSERT INTO orders
VALUES (60001, '2024-01-10', '2024-01-20', NULL, 'In Process', 103);

INSERT INTO orders
VALUES (60002, '2024-01-11', '2024-01-21', NULL, 'On Hold', 103);

INSERT INTO orders
VALUES (60003, '2024-01-12', '2024-01-22', NULL, 'Shipped', 103);

-- FAIL
INSERT INTO orders
VALUES (60004, '2024-01-13', '2024-01-23', NULL, 'In Process', 103);
```

Exercise 3

We want to ensure that every order placed has stock available in the inventory.
Before inserting an order detail into the `orderdetails` table, verify that the product is in stock and that the requested quantity does not exceed the available stock (`quantityInStock`).
If the product is not in stock or the requested quantity exceeds the stock, the trigger must throw an error with a descriptive message (different for each case). Demonstrate how to test the trigger.

Examples:

#	Time	Action	Message	Duration / Fetch
1	20:16:01	INSERT INTO orderdetails (orderNumber, productCode, quantityOrdered, priceEach, orderLineNumber) ...	1 row(s) affected	0.016 sec
2	20:16:04	INSERT INTO orderdetails (orderNumber, productCode, quantityOrdered, priceEach, orderLineNumber) ...	Error Code: 1644. ERROR: Insufficient stock to fulfill the order	0.000 sec
3	20:16:06	INSERT INTO orderdetails (orderNumber, productCode, quantityOrdered, priceEach, orderLineNumber) ...	Error Code: 1644. ERROR: The product must be in stock to place an order	0.000 sec

Problem:

You cannot:
Order a product with no stock
Order more than available stock

Solution:

BEFORE INSERT trigger on `orderdetails`

Explanation:

1. Check if the product exists in stock
2. Check if the requested quantity is available

Different error messages:

- "Product not in stock"
- "Insufficient stock"

Task for BD06

Farré Vicente Josep

Bases de dades

```
DROP TRIGGER IF EXISTS before_insert_orderdetails;

DELIMITER $$

CREATE TRIGGER before_insert_orderdetails
BEFORE INSERT ON orderdetails
FOR EACH ROW
BEGIN
    DECLARE stock INT;

    SELECT quantityInStock
    INTO stock
    FROM products
    WHERE productCode = NEW.productCode;

    IF stock IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Product does not exist';

    ELSEIF stock = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Product out of stock';

    ELSEIF stock < NEW.quantityOrdered THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Insufficient stock';
    END IF;
END$$

DELIMITER ;

-- TESTS

-- SUCCESS
INSERT INTO orderdetails
VALUES (10100, 'S10_1678', 2, 95.00, 1);

-- FAIL (too many units)
INSERT INTO orderdetails
VALUES (10100, 'S10_1678', 99999, 95.00, 2);

-- FAIL (invalid product)
INSERT INTO orderdetails
VALUES (10100, 'INVALID_CODE', 5, 50.00, 3);
```

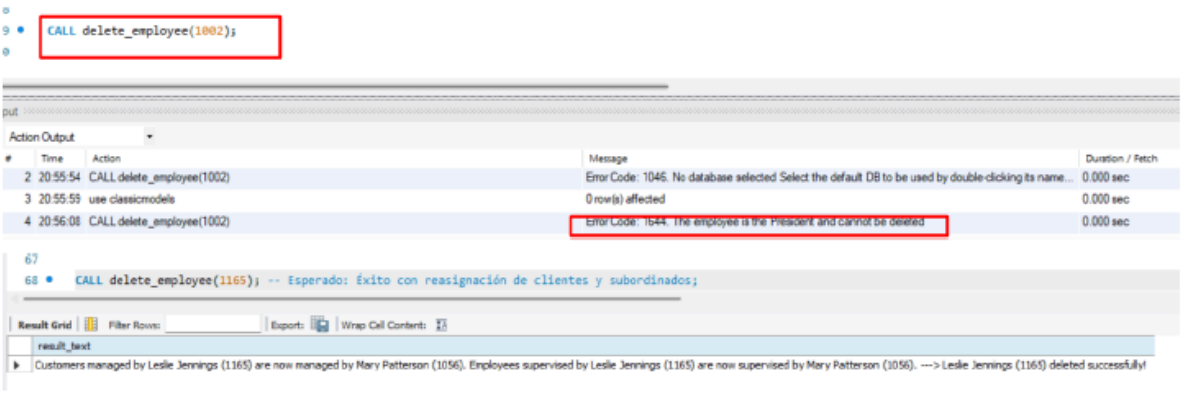
Task for BD06
Farré Vicente Josep
Bases de dades

Exercise 4

Create a procedure named `delete_employee` to delete an employee from the `employees` table in `classicmodels`. Before deletion, ensure the following conditions are met:

1. Customers assigned to the employee must be reassigned to the employee's supervisor.
2. Employees supervised by the employee must be reassigned to the employee's supervisor.
3. If the employee has no supervisor (i.e., they are the president), they cannot be deleted, and a custom error must be returned.
   The procedure must accept the `employeeNumber` as a parameter and return a custom error if the `employeeNumber` does not exist.

Examples:



Problem:
Delete an employee while ensuring:
1. Customers are reassigned
2. Subordinates are reassigned
3. The president cannot be deleted
4. Handle non-existing employees

Solution:
Stored procedure `delete_employee`

Step-by-step explanation:
- Retrieve employee and supervisor
- If not found → error
- If no supervisor → president → error
- Otherwise:
  - Reassign customers
  - Reassign employees
  - Delete employee

Task for BD06

Farré Vicente Josep

Bases de dades

```
DROP PROCEDURE IF EXISTS delete_employee;

DELIMITER $$

CREATE PROCEDURE delete_employee (IN pid INT)
BEGIN
    DECLARE emp_name VARCHAR(200);
    DECLARE boss_id INT;
    DECLARE boss_name VARCHAR(200);

    SELECT CONCAT(firstName, ' ', lastName), reportsTo
    INTO emp_name, boss_id
    FROM employees
    WHERE employeeNumber = pid;

    IF emp_name IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee does not exist';

    ELSEIF boss_id IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'The president cannot be deleted';

    ELSE
        UPDATE customers
        SET salesRepEmployeeNumber = boss_id
        WHERE salesRepEmployeeNumber = pid;

        UPDATE employees
        SET reportsTo = boss_id
        WHERE reportsTo = pid;

        DELETE FROM employees
        WHERE employeeNumber = pid;

        SELECT CONCAT(firstName, ' ', lastName)
        INTO boss_name
        FROM employees
        WHERE employeeNumber = boss_id;

        SELECT CONCAT(
            'Employee ', emp_name,
            ' deleted. Reassigned to ', boss_name
        ) AS message;
    END IF;
END$$

DELIMITER ;

-- TESTS

CALL delete_employee(9999); -- FAIL
CALL delete_employee(1002); -- FAIL
CALL delete_employee(1165); -- SUCCESS
```

Task for BD06
Farré Vicente Josep
Bases de dades

Exercise 5

Create a procedure named `customers_sales_report` that accepts three parameters:

1. The office name (`office_name`) as an input parameter.
2. The year (`year_param`) as an input parameter.
3. An output parameter that returns a detailed report of monthly sales for the employees of that office during the specified year (based on `orderDate`).

The report must include:

- The full name of the employee.
- The customer's name (`customerName`).
- The total sales made by the employee for that customer during the year.

If no sales are found, the procedure must return a message indicating that no data is available.

Problem:
Generate a sales report based on:
- Office
- Year

Solution:
Procedure `customers_sales_report` with:
- 2 input parameters
- 1 output parameter

Explanation:

- Uses a **cursor**
- Iterates through grouped results
- Builds a report including:
  - Employee
  - Customer
  - Total sales

```
DROP PROCEDURE IF EXISTS customers_sales_report;

DELIMITER $$

CREATE PROCEDURE customers_sales_report(
    IN office_name VARCHAR(50),
    IN year_param INT,
    OUT sales_report TEXT
)
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE line TEXT;

    DECLARE cur CURSOR FOR
    SELECT CONCAT(
```

Task for BD06
Farré Vicente Josep
Bases de dades

```
        'Month: ', MONTH(o.orderDate),
        ' | Employee: ', e.firstName, ' ', e.lastName,
        ' | Customer: ', c.customerName,
        ' | Total: ', FORMAT(SUM(od.quantityOrdered * od.priceEach),2)
    )
    FROM employees e
    JOIN offices o2 ON e.officeCode = o2.officeCode
    JOIN customers c ON e.employeeNumber = c.salesRepEmployeeNumber
    JOIN orders o ON c.customerNumber = o.customerNumber
    JOIN orderdetails od ON o.orderNumber = od.orderNumber
    WHERE o2.city = office_name
      AND YEAR(o.orderDate) = year_param
    GROUP BY MONTH(o.orderDate), e.employeeNumber, c.customerNumber;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    SET sales_report = '';

    OPEN cur;

    read_loop: LOOP
        FETCH cur INTO line;
        IF done THEN
            LEAVE read_loop;
        END IF;

        SET sales_report = CONCAT(sales_report, line, '\n');
    END LOOP;

    CLOSE cur;

    IF sales_report = '' THEN
        SET sales_report = 'No sales data available';
    END IF;
END$$

DELIMITER ;

-- TESTS
CALL customers_sales_report('San Francisco', 2003, @report);
SELECT @report;

CALL customers_sales_report('Madrid', 2025, @report);
SELECT @report;
```

Exercise 6

Create a function that takes a `customerNumber` (customer ID) as a parameter and returns a string summarizing the number of orders and products purchased by that customer.

Task for BD06

Farré Vicente Josep

Bases de dades

Problem:
Show a summary of:

- Number of orders
- Number of products

Solution:

Function `customer_summary`

Explanation:

- Count total orders
- Count distinct products
- Return a formatted string

Key sentence:

"Functions allow reusable logic and simplify queries."

Result:

- Customer with orders → summary
- No orders → specific message

```
DROP FUNCTION IF EXISTS customer_summary;

DELIMITER $$

CREATE FUNCTION customer_summary (customer_id INT)
RETURNS VARCHAR(255)
READS SQL DATA
BEGIN
    DECLARE orders_count INT DEFAULT 0;
    DECLARE products_count INT DEFAULT 0;

    SELECT COUNT(*)
    INTO orders_count
    FROM orders
    WHERE customerNumber = customer_id;

    IF orders_count = 0 THEN
        RETURN 'No orders';
    END IF;

    SELECT COUNT(DISTINCT productCode)
    INTO products_count
    FROM orderdetails od
    JOIN orders o ON od.orderNumber = o.orderNumber
    WHERE o.customerNumber = customer_id;

    RETURN CONCAT(
        orders_count, ' orders and ',
        products_count, ' different products'
    );
```

Task for BD06

Farré Vicente Josep

Bases de dades

```
END$$

DELIMITER ;

-- TESTS

SELECT customer_summary(103);
SELECT customer_summary(999);

SELECT customerNumber, customerName, customer_summary(customerNumber) AS summary
FROM customers
LIMIT 10;
```